SLMForge

Milestone 3: Task Type System

Summer Profile Building Drive 2026

Newton School of Technology

Section 1: Before We Begin

Where We Are

SLMForge fine-tunes a small language model (SLM) from your own data - the promise is 'just point at your data and get a trained model'. In M2 you built the data layer: ingest and normalise datasets into clean records. But to train and evaluate a model, the engine must know **what kind of task** the data represents - is it classification? summarisation? question answering? The user should not have to spell that out. M3 is the **Task Type System** that figures it out.

What Will You Build in Milestone 3?

M3 auto-detects the task type, then wires the right prompt template and the right evaluation metric for that type - while always letting the user override. Five task types are supported: **classification, summarisation, QA, instruction, chat**. Five issues:

Issue	Title	Key Deliverable
#10	Task detector v1	Heuristics + small classifier -> type + confidence + alternatives
#11	Per-task templates	One canonical prompt/target template per task type
#12	Per-task metric selector	Map task type -> the right metric suite
#13	User-override path (CLI + UI)	--task / --base / --template flags and dropdowns
#14	Detector regression suite	30+ labelled samples/type; CI gates on accuracy

Samajhne ke liye ek analogy: Darzi jo Naap khud Samajh le

Socho ek expert darzi hai. Tum kapda le ke jaate ho, woh dekhte hi samajh jaata hai 'yeh shirt ka hai ya pant ka' (task detector, #10). Har cheez ka ek fixed silai-pattern hai (per-task template, #11), aur har cheez ki fitting alag tareeke se naapi jaati hai (per-task metric, #12). Par agar darzi galat samajh le, tum bol sakte ho 'nahi, isse kurta banao' (user override, #13). Aur darzi apne kaam ko regularly check karta hai taaki galti na badhe (regression suite, #14). SLMForge yehi darzi hai - user ko naap-jokh ki tension nahi deta.

Section 2: Task Detector (Issue #10)

The whole 'just point at data' UX rests on the detector picking the right task type most of the time. It combines cheap **heuristics** on the dataset's shape and content with a small classifier, and returns not just a label but a **confidence** and **alternative suggestions** so the UI can show a sensible default that the user can change.

What signals the heuristics use

Task type	Tell-tale signals
Classification	A small fixed set of repeated labels in the target column
Summarisation	Target much shorter than input; high overlap of target words with input
QA	A question field + a short answer field, often with a context passage
Instruction	An 'instruction' (+ optional 'input') field mapping to a response
Chat	A list of role-tagged turns (user / assistant) per record

```
# src/slmforge/task/detector.py
from dataclasses import dataclass

@dataclass
class Detection:
    task_type: str
    confidence: float           # 0..1
    alternatives: list[tuple]   # [(task_type, score), ...]

def detect(dataset) -> Detection:
    scores = {
        'classification': few_repeated_labels(dataset),
        'summarisation':  target_shorter_and_overlapping(dataset),
        'qa':             has_question_answer_fields(dataset),
        'instruction':    has_instruction_response(dataset),
        'chat':           has_role_tagged_turns(dataset),
    }
    ranked = sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
    top, conf = ranked[0]
    return Detection(top, conf, ranked[1:])   # confidence + alternatives
```

The target is **>= 85% accuracy** on the regression suite (#14). Because it also returns confidence and runner-ups, a low-confidence detection can prompt the user to confirm rather than silently proceeding on a guess.

Section 3: Per-Task Templates (Issue #11)

One engine has to train and serve all five task types, so each type needs a **canonical prompt shape**. A template turns a raw record into a (prompt, target) pair the model can learn from, and a renderer applies it. The templates must align with real chat formats - **Phi-3** and **Llama 3.1** - so the same records serve either base model.

The round-trip property

A key correctness test: rendering a record into a prompt and then stripping the template back out must return the original content (`render -> strip == original`). If it does not, the template is corrupting data - silently changing what the model sees versus what you think it sees.

```
# src/slmforge/task/templates.py
TEMPLATES = {
  'classification': '{input}\nLabel:',                # target = the label
  'summarisation':  'Summarize:\n{input}\nSummary:',   # target = summary
  'qa':             'Context: {context}\nQ: {question}\nA:',
  'instruction':    '{instruction}\n{input}\nResponse:',
  'chat':           '<|user|>{user}<|assistant|>',      # Phi-3 / Llama fmt
}

def render(record, task_type) -> tuple[str, str]:
    prompt = TEMPLATES[task_type].format(**record)
    return prompt, record['target']          # (prompt, target) pair
```

> **What if we skip this?**
>
> If every user invents their own template, the engine cannot guarantee the prompt matches the base model's expected chat format - training silently degrades, and results are not comparable across runs. One canonical template per task keeps training reproducible and lets you swap Phi-3 for Llama 3.1 without rewriting prompts.

Section 4: Per-Task Metric Selector (Issue #12)

Evaluation must 'just work' - the user should not hand-pick metrics. The selector maps each task type to the metric suite that actually makes sense for it, behind a uniform `metric(preds, refs)` call so the eval harness treats them all the same.

Task type	Metric suite	Why
Classification	Accuracy, F1 (macro)	Discrete labels; F1 handles class imbalance
Summarisation	ROUGE-1/2/L	Overlap of generated vs reference summary
QA	Exact match, token F1	Answer correctness, exact and partial
Instruction	ROUGE / win-rate	Open-ended response similarity or judged quality
Chat	Perplexity / judge score	Fluency and helpfulness of replies

The point is not a generic menu of every metric - it is a **curated, correct** suite per task, chosen automatically. A plug-in interface lets you register a new metric later without touching the eval core, and every metric exposes the same call signature so the harness stays task-agnostic.

```
# src/slmforge/task/metrics.py
METRIC_SUITES = {
  'classification': [accuracy, f1_macro],
  'summarisation':  [rouge1, rouge2, rougeL],
  'qa':             [exact_match, token_f1],
  'instruction':    [rougeL, win_rate],
  'chat':           [perplexity, judge_score],
}
def suite_for(task_type):  return METRIC_SUITES[task_type]   # non-empty
```

> **Analogy: Har Khel ka Apna Scoreboard**
>
> Cricket run se, football goal se, chess move se jeeta jaata hai - ek hi scoreboard sab pe nahi chalta. Metric selector wahi karta hai: task type dekh ke sahi scoreboard automatically laga deta hai. User ko 'kaunsa metric?' sochna nahi padta - classification pe accuracy/F1, summarisation pe ROUGE, apne aap.

Section 5: User-Override Path (Issue #13)

The detector will sometimes be wrong, and users must be able to correct it **without editing code**. So the task type, base model, and template are all overridable from both the CLI and the UI, flowing through the same validated API surface.

How overrides flow

- **CLI flags.** `--task`, `--base`, `--template` let a user pin any of the three explicitly.
- **UI dropdowns.** Same three choices, with the detector's suggestion pre-selected as the default - so the common case is one click, and the override is right there.
- **Server-side validation.** Invalid values (unknown task, incompatible template) are rejected with a clear reason - the UI and CLI hit the identical API, so validation lives in one place.

```
# extends src/slmforge/cli/main.py and api/schemas.py
slmforge build --task classification --base phi-3 \
               --template classification --auto --recipe sanity

# schema validation (server-side, shared by CLI + UI)
VALID_TASKS = {'classification','summarisation','qa','instruction','chat'}
if task not in VALID_TASKS:
    raise ValueError(f'unknown task {task!r}; choose one of {VALID_TASKS}')
```

> **Try This**
>
> Force an override end-to-end and confirm it reaches the engine: `slmforge build --task classification --auto --recipe sanity`. Then pass a bad value like `--task translate` and confirm it is rejected with a readable reason, not a stack trace.

Section 6: Detector Regression Suite (Issue #14)

The detector is the foundation of the UX, so it must not silently get worse as you add task coverage. The regression suite is a labelled test set that **gates CI**: if detector accuracy drops below the threshold, the build fails.

What it contains

- **30+ labelled samples per task type** in `tests/fixtures/task_detection/` - real dataset shapes with their known-correct task type.
- **A CI gate**: overall accuracy must stay **>= 85%**; the pytest run fails below that, so a regression cannot merge.
- **A clear extension path**: adding a new task type means adding its heuristic, its template, its metric suite, and a fresh batch of 30+ labelled fixtures.

```
# tests/integration/test_task_detector_regression.py
def test_detector_accuracy():
    samples = load_fixtures('tests/fixtures/task_detection/')
    correct = sum(detect(s.dataset).task_type == s.label for s in samples)
    acc = correct / len(samples)
    assert len(samples) >= 30 * 5          # >= 30 per type, 5 types
    assert acc >= 0.85, f'detector regressed to {acc:.2%}'
```

> **What if we skip this?**
>
> Without a regression gate, a well-meaning tweak to one heuristic can quietly wreck detection for another task type, and nobody notices until a user's classification dataset gets trained as chat. The CI gate is what turns 'it worked on my machine' into 'it provably still works'.

Section 7: The Task Type System

The five issues form one system: detect the type, render with the matching template, evaluate with the matching metric, let the user override any of it, and guard the whole thing with a regression suite.

Step	Component	Role
1	Detector (#10)	Reads the dataset -> task type + confidence + alternatives
2	Templates (#11)	Render records into (prompt, target) for the detected type
3	Metric selector (#12)	Pick the right metric suite for that type
4	Override path (#13)	Let the user pin task / base / template via CLI or UI
5	Regression suite (#14)	Gate CI so the detector never silently regresses

Dependency chain

Issue	Depends On
#10 Task detector	#7 (data layer, M2)
#11 Per-task templates	#10
#12 Per-task metrics	#11
#13 User-override path	#10

#14 Regression suite	#10

Definition of done for M3

- Detector returns task type + confidence + alternatives; >= 85% accuracy on the suite.
- One canonical template per task; render -> strip round-trips to the original; aligns with Phi-3 / Llama 3.1.
- Per-task metric selector returns a non-empty suite with a uniform call shape.
- CLI `--task` flag and UI dropdown both flow through to the engine, with validation.
- Regression suite has 30+ samples per type and fails CI below 85%.

Section 8: Action Plan

Day	Focus	Issue	Verify
1	Detector heuristics + confidence + alternatives	#10	>=85% on fixtures; returns runner-ups.
2	Per-task templates + round-trip renderer	#11	render->strip equals original; Phi-3/Llama aligned.
3	Metric selector + plug-in interface	#12	Each type returns a non-empty suite; uniform call.
4	Override CLI/UI + regression suite + CI gate	#13, #14	Overrides validated; CI fails below 85%.

Build the detector first - templates, metrics, overrides, and the regression suite all hang off it. The regression fixtures (#14) are worth writing early; they double as your development test set for #10.

Section 9: Defense Questions

Q1: Walk through the detector heuristics. Why these signals?

Each task type has a structural fingerprint: classification shows a small set of repeated labels; summarisation has targets much shorter than inputs with high word overlap; QA has question/answer (and often context) fields; instruction has an instruction-response shape; chat has role-tagged turns. These signals are cheap to compute and map directly to how the data is organised, which is why they are reliable before any model is trained.

Q2: What does the regression suite look like? How would you add a new task type?

It is 30+ labelled dataset samples per task type under `tests/fixtures/task_detection/`, with a pytest that computes overall accuracy and fails CI below 85%. Adding a task type means: add its detection heuristic, add a canonical template, add its metric suite, and add 30+ labelled fixtures so the gate covers it too.

Q3: Why one canonical template per task type? What breaks if users define their own?

One template per task keeps training reproducible and guarantees the prompt matches the base model's expected chat format (Phi-3 / Llama 3.1). If everyone rolls their own, prompts drift from the model's format, results stop being comparable across runs, and swapping base models means rewriting prompts. Users can still override to a different canonical template - they just do not hand-craft the format.

Q4: How is the per-task metric selector different from a generic metric list?

A generic list makes the user choose (and choose wrongly - ROUGE on a classification task is meaningless). The selector maps each task type to a curated, correct suite automatically - accuracy/F1 for classification, ROUGE for summarisation - behind a uniform `metric(preds, refs)` call, so eval 'just works' without the user knowing which metric fits which task.

Q5: What does an override look like in docs/ENGINE_API.md?

It documents the three override parameters - `task`, `base`, `template` - exposed identically via CLI flags and the API schema, with the detector's suggestion as the default and server-side validation rejecting invalid combinations with a reason. The doc shows a build call that pins the task explicitly and the validation errors an invalid value produces.

Section 10: Glossary

Term	Meaning
SLM	Small Language Model - the compact model SLMForge fine-tunes.
Task type	The kind of task a dataset represents: classification, summarisation, QA, instruction, chat.
Task detector	Heuristics + classifier that infers task type with confidence + alternatives.
Confidence	The detector's certainty (0-1); low confidence prompts user confirmation.
Template	The canonical prompt/target shape for a task type; renders a record into (prompt, target).
Round-trip	render -> strip must equal the original record - proof the template is lossless.
Phi-3 / Llama 3.1	The supported base models; templates align to their chat formats.
Metric suite	The set of metrics correct for a task (accuracy/F1, ROUGE, exact match, etc.).
ROUGE	Overlap metric for summarisation (n-gram / longest-common-subsequence).
Override	User pinning task / base / template via CLI flag or UI dropdown.
Regression suite	Labelled fixtures that gate CI so detector accuracy cannot silently drop.
CI gate	A test that fails the build when accuracy falls below the threshold (85%).

That is M3. SLMForge can now look at raw data and know what to do with it: detect the task, apply the right template and metric automatically, honour a user override when the guess is wrong, and guard the detector with a regression gate - the machinery that makes 'just point at your data' actually hold up.